

Arquitectura y consideraciones para un copiloto de ingeniería

Propuesta de MVP plug-and-play para matemáticas avanzadas e ingeniería aplicada, con infraestructura de bajo costo y routing de modelos.

Objetivo de diseño

Crear una experiencia tipo chat donde el estudiante no tenga que escoger modelos. La plataforma selecciona estrategia, modelo, herramientas de cálculo y formato de salida de manera transparente.

Escala inicial: 1 usuario; esperable <10 y máximo cercano a 20

Frontend: Angular 21.2.6

Backend: Node.js 24.14.1 + TypeScript

Prioridad: Calidad de respuesta, verificación y simplicidad operativa

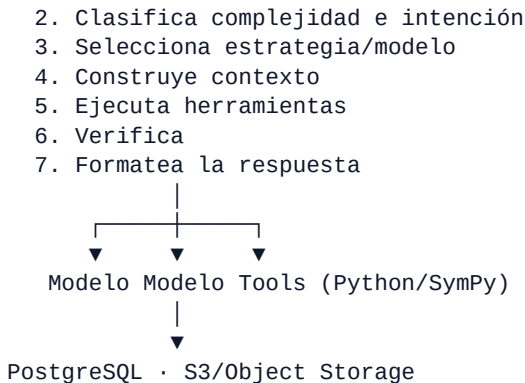
Principios clave

- El valor principal no está en “usar el mejor modelo”, sino en escoger la mejor estrategia para cada consulta.
- La capa de IA debe ser agnóstica del proveedor y trabajar con perfiles como FAST, MATH, DEEP_REASONING o DOCUMENT.
- Para matemáticas e ingeniería, herramientas deterministas como Python, SymPy, NumPy y SciPy son esenciales para verificar resultados.
- LaTeX/Markdown deben tratarse como formatos nativos; PDF y otros archivos se generan desde contenido estructurado.
- Con menos de 20 usuarios conviene una arquitectura monolítica/modular sencilla, sin microservicios, Kubernetes ni mensajería compleja.

1. Arquitectura conceptual

La aplicación puede dividirse en cuatro capas: interfaz, API, orquestación de IA y persistencia/herramientas. El frontend nunca debería necesitar saber qué modelo respondió.





2. Estrategia antes que modelo

La pregunta correcta no es “¿qué modelo es mejor?”, sino “¿qué combinación de modelo + herramientas + verificación resuelve mejor este problema?”.

Consulta	Estrategia sugerida
Demostración matemática	Modelo de razonamiento + prompt de prueba formal.
Autovalores de una matriz grande	Modelo de razonamiento + SymPy/Python para verificación.
EDO con análisis numérico	Modelo + SciPy + Matplotlib + interpretación final.

3. Abstracción de proveedores

Evita acoplar el sistema a un SDK específico. Define una interfaz propia y adaptadores por proveedor.

```
interface AIProvider {
  generate(request: AIRequest): Promise<AIResponse>;
}

// Adaptadores posibles
OpenAIProvider
AnthropicProvider
GeminiProvider
OpenRouterProvider
```

La aplicación habla con un AIService/Orchestrator; solo los adaptadores conocen los detalles de cada proveedor.

4. Model Router y perfiles

Separar “perfil” de “modelo” permite cambiar proveedores sin tocar el dominio. Los perfiles son capacidades estables; los modelos concretos son configuración.

```
profiles:
  default:
    primary: model-A
    fallback: model-B
  math:
    primary: model-C
    fallback: model-D
```

```

deep_reasoning:
  primary: model-E
  fallback: model-C

```

Perfiles iniciales sugeridos: DEFAULT, FAST, MATH, DEEP_REASONING, DOCUMENT, VISION y CODE.

5. Herramientas deterministas para matemáticas

En un producto académico, confiar solo en el texto del LLM es insuficiente. Para cálculos verificables conviene integrar un sandbox con Python.

- SymPy: álgebra simbólica, derivadas, integrales, límites y ecuaciones.
- NumPy: matrices y cálculo numérico.
- SciPy: EDO, optimización y métodos científicos.
- Matplotlib: gráficos y visualización reproducible.

El sandbox debe limitar CPU, RAM, tiempo de ejecución, sistema de archivos y acceso a red. Nunca ejecutes código generado por el modelo dentro del proceso principal de Node.js.

6. Pipeline del agente

Un flujo robusto puede funcionar de la siguiente manera:

```

UserPrompt
  ↓
IntentClassifier
  ↓
ContextBuilder
  ↓
StrategySelector
  ↓
Model + Tools
  ↓
Verifier
  ↓
ResponseFormatter
  ↓
Usuario

```

El verificador debería activarse solo para consultas complejas, porque agrega costo y latencia. Puede revisar errores algebraicos, pasos injustificados y discrepancias entre el razonamiento y el resultado calculado.

7. Prompts como parte del producto

Mantén prompts versionados y modulares, por ejemplo: base.prompt, math.prompt, proof.prompt, engineering.prompt y document-analysis.prompt. El orquestador combina solo los módulos relevantes.

Para problemas matemáticos, el comportamiento esperado debería incluir: declarar supuestos, mencionar definiciones/teoremas usados, no saltar pasos no triviales, usar LaTeX, separar intuición de prueba formal y verificar cálculos cuando sea posible.

8. Intenciones o “modos” implícitos

No es necesario exponerlos al usuario. El router puede inferir intenciones como EXPLAIN, SOLVE, PROVE, VERIFY, COMPUTE, SUMMARIZE, ANALYZE_DOCUMENT y CODE. Esto permite responder distinto a “¿qué es una función holomorfa?” y a “demuestra que $f(z)=z^2$ es holomorfa”.

9. LaTeX, Markdown y artefactos

LaTeX debe ser de primera clase. Una opción razonable es almacenar Markdown con expresiones LaTeX y renderizarlo en Angular mediante KaTeX. Evita almacenar HTML generado como fuente principal.

Modela la respuesta como contenido + artefactos + metadatos, no como un string plano:

```
interface AssistantResponse {
  markdown: string;
  artifacts?: Artifact[];
  metadata: {
    model: string;
    latencyMs: number;
    tokens: number;
    verified: boolean;
  };
}
```

PDF, Markdown y LaTeX deberían generarse desde la misma respuesta estructurada. El modelo produce contenido; tu aplicación produce los archivos.

10. Conversaciones y contexto

No envíes todo el historial indefinidamente. Para chats largos combina: System Prompt + resumen de conversación + mensajes anteriores relevantes + mensajes recientes + prompt actual.

Cuando el historial crece, resume bloques antiguos y conserva literalmente una ventana reciente. Para esta escala, PostgreSQL es suficiente.

11. Modelo de datos mínimo

Tablas iniciales recomendadas: users, conversations, messages, attachments, model_executions y artifacts. model_executions debería registrar proveedor, modelo, tokens de entrada/salida, latencia, costo estimado, herramientas usadas, reintentos, fallback y resultado de verificación.

12. Observabilidad y feedback

Aunque haya un solo usuario, registra desde el primer día: prompt/configuración, modelo, tokens, latencia, costo, tool calls, errores, retries y fallbacks. Esto permitirá ajustar el router con evidencia real.

Añade feedback simple 👍/👎 y, para respuestas negativas, categorías como “resultado incorrecto”, “explicación confusa”, “faltaron pasos” y “formato incorrecto”. Ese dataset será más útil que optimizar solo con benchmarks públicos.

13. Archivos y RAG

No incluiría RAG avanzado en el MVP. Sí soportaría carga de PDFs/imágenes temprano, porque habilita flujos de alto valor: explicar una guía, resolver un ejercicio concreto o respetar únicamente los teoremas entregados por el estudiante.

Cuando aparezca la necesidad de consultar apuntes extensos o múltiples documentos, entonces incorpora parsing, chunking, embeddings y búsqueda semántica.

14. Tutor vs. solucionador

Distingue entre “resuélvelo” y “ayúdame a resolverlo”. El primer caso entrega una solución completa; el segundo puede guiar mediante preguntas y pistas progresivas. Esta diferenciación puede ser una de las características educativas más valiosas del producto.

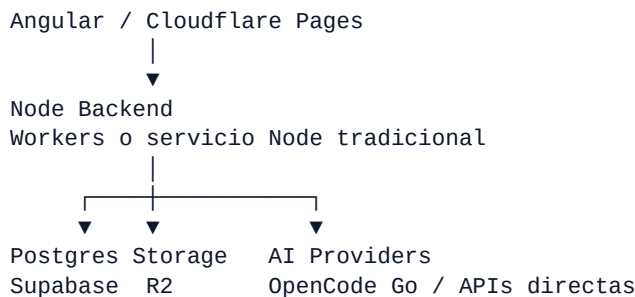
15. Infraestructura: AWS no es obligatorio

Para 1-10 usuarios (y un máximo cercano a 20), el MVP puede operar con servicios gratuitos o con free tiers. El objetivo debería ser minimizar complejidad y costo fijo; AWS puede quedar como una opción futura o como plataforma de aprendizaje, no como requisito técnico.

Una combinación simple y de bajo costo puede ser:

- Frontend Angular: Cloudflare Pages.
- Backend Node.js: Cloudflare Workers si el backend es liviano, o un servicio de contenedores/Node tradicional si necesitas mayor compatibilidad.
- Base de datos: Supabase PostgreSQL o Neon PostgreSQL.
- Archivos: Supabase Storage o Cloudflare R2.
- Sandbox Python/SymPy: proceso o contenedor aislado, separado del backend principal.

Cloudflare mantiene un plan gratuito para Workers/Pages con límites de uso; Supabase también ofrece un plan Free. Los límites y precios pueden cambiar, por lo que conviene tratarlos como configuración operativa y no como supuestos permanentes.



16. Suscripciones de chat vs. API de modelos

Una suscripción de usuario final no implica automáticamente uso de API para tu aplicación. En OpenAI, la facturación de ChatGPT y la plataforma API son separadas; para invocar modelos desde tu backend mediante la API, se usa una cuenta/API con su propia facturación y credenciales.

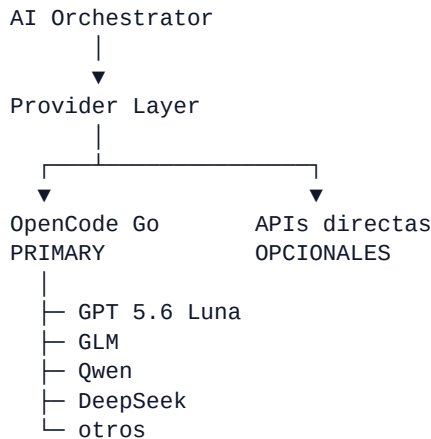
Regla de diseño: nunca dependas de que el usuario final tenga una suscripción específica. El backend debe poseer o recibir las credenciales del proveedor que corresponda, según el modelo operativo que elijas.

17. OpenCode Go como proveedor inicial

OpenCode Go puede ser especialmente útil para el MVP porque entrega una API key y acceso a varios modelos mediante una suscripción de bajo costo. La lista de modelos es dinámica y puede cambiar.

Al 1 de octubre de 2026, la documentación de OpenCode Go indica un precio regular de USD 10/mes (USD 5 el primer mes) y una lista que incluye, entre otros, GPT 5.6 Luna, GLM, Kimi, Qwen, DeepSeek, MiniMax y MiMo.

Esto permite comenzar con un único proveedor lógico y experimentar con diferentes modelos sin integrar cada fabricante desde el primer día.



Antes de abrir el producto a terceros, revisa los términos vigentes del proveedor para confirmar que el patrón de uso de tu aplicación y la distribución del acceso están permitidos.

18. Límites de OpenCode Go y control de consumo

Los límites de Go se expresan en valor de uso, no en un número fijo de consultas. Para modelos con límite mensual de USD 60, la documentación muestra como referencia: USD 12 por ventana de 5 horas, USD 30 por semana y USD 60 por mes. Algunos modelos tienen límites mensuales menores.

El número real de peticiones depende del modelo y del tamaño de cada consulta. Por ejemplo, la documentación estima miles de solicitudes para GPT 5.6 Luna y muchas más para modelos de menor costo, pero estas cifras son aproximadas y pueden cambiar.

Tu aplicación debería aplicar además sus propios límites, independientemente del proveedor:

- requests por usuario/día y por mes;
- tokens de entrada/salida;
- costo equivalente o unidades de consumo;
- rate limits para evitar loops o bugs;
- reintentos y fallbacks registrados en observabilidad.

19. Fallbacks y estrategia de costo

Para el MVP, evita fallbacks que generen gasto automáticamente. Si OpenCode alcanza cuota o rate limit, es válido informar que el servicio está temporalmente limitado. Cuando el producto madure, puedes agregar un proveedor directo como fallback controlado.

OpenCode disponible → usarlo

OpenCode limitado → error controlado / aviso

Fase posterior:

OpenCode limitado → proveedor secundario con presupuesto máximo

También puedes usar modelos locales (por ejemplo, vía Ollama) para tareas baratas como clasificación, resumen o routing, dejando el razonamiento matemático exigente para modelos remotos más capaces.

20. Stack propuesto para el MVP actualizado

Área	Tecnología / opción inicial
Frontend	Angular 21.2.6, TypeScript, KaTeX, Markdown; Cloudflare Pages
Backend	Node.js 24.14.1 + TypeScript; Workers o servicio Node tradicional
Datos	PostgreSQL; Supabase o Neon
IA	AIProvider + router/perfiles; OpenCode Go como proveedor inicial
Cálculo	Sandbox Python con SymPy, NumPy, SciPy, Matplotlib
Storage	Supabase Storage o Cloudflare R2
Exportación	Markdown, LaTeX y PDF
Costo objetivo	Infraestructura cercana a USD 0; IA como costo principal

21. Benchmark propio para el Model Router

Aprovechar varios modelos bajo un mismo proveedor permite construir un benchmark propio. En lugar de decidir por reputación general, evalúa con ejercicios reales de tus dominios objetivo.

Ejemplo de dataset: 50-100 ejercicios distribuidos entre Análisis Real, Variable Compleja, Álgebra Lineal, EDO y Cálculo Numérico. Para cada modelo mide correctitud, calidad de demostración, uso de LaTeX, latencia, necesidad de herramientas y costo equivalente.

Con esos datos puedes mapear perfiles a modelos concretos: FAST → modelo económico; DEFAULT → modelo equilibrado; MATH → modelo con mejor desempeño matemático validado por tu benchmark.

22. Alcance recomendado del MVP

1. Chat con streaming.
2. Markdown + LaTeX.
3. Historial de conversaciones.
4. Abstracción AIProvider.
5. OpenCode Go como proveedor inicial, sin acoplar el dominio a su SDK.
6. 2-3 perfiles iniciales: FAST, DEFAULT y MATH.
7. Router básico por reglas.
8. Sandbox Python/SymPy.
9. Carga de PDF e imágenes.
10. Exportación .md y .pdf.

11. Telemetría de tokens, consumo, modelo y latencia.
12. Límites internos por usuario.
13. Feedback 👍/👎 y dataset de evaluación.

23. Lo que dejaría fuera del primer MVP

RAG avanzado, agentes multi-step muy complejos, vector DB, colaboración, organizaciones, fine-tuning, microservicios, fallbacks de pago automáticos y optimizaciones de gran escala. El objetivo inicial es validar calidad, costo y consistencia con estudiantes reales.

Referencias operativas (consultadas en octubre de 2026)

- OpenCode Go - documentación oficial de modelos, precio y límites de uso: opencode.ai/docs/go/
- Cloudflare Workers/Pages - documentación oficial de pricing y free plan: developers.cloudflare.com/workers/platform/pricing/
- Supabase - pricing y documentación de Free Plan: supabase.com/pricing
- OpenAI Help Center - la facturación de ChatGPT y API se administra por separado: help.openai.com/

Conclusión

El diferenciador no será el chat ni la infraestructura. Será la capa de orquestación que decide cómo resolver una consulta académica: qué perfil usar, cuándo ejecutar herramientas, cómo verificar el resultado y cómo presentarlo con rigor pedagógico.